A 215-State ZFC Consistency Machine via Conditional Origin Markers

A new exact machine, a two-head transfer calculus, and a correction to an experimental return path

ChatGPT-assisted research for Vladimir Reshetnikov's ProveIt repository

21 October 2026

Abstract

We construct an explicit deterministic binary Turing machine CM_{215} with 215 working states, including initialization, and a separate halt. Its native transition table differs in six rows from the accepted K_{215} at ProveIt commit `75716498437bfc7291d5900cab2983a11fd54bac`. An ordinary mathematical proof establishes, for every initial binary tape from the designated initial state, halting equivalence of the two machines. Consequently the inherited consistency calibration gives

$$\mathsf{ZFC} \vdash \big(\neg\,\mathrm{Halt}(\mathsf{CM}_{215}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{ZFC})\big).$$

The new construction merges two decoder roles and reuses the released label as a marker-reading helper. The origin invariant is required only on the symbol on which the old roles disagree; a seemingly incompatible incoming edge is harmless because it forces the other symbol. This yields a materially different 215-state graph, independently distinguished from 77 prior tables, and a short whole-tape correctness argument. Fresh symbolic verification discharges all 6,176 primitive cases; an independent finite replay checks the complete 95,714-step blank prefix.

On the formalism side, we prove completeness and a uniform protected-stack bound for a three-command language whose transfer removes two stack heads. Five composite and five direct source variants have 202 distinct subtrees; the direct command uses 61 primitives instead of 104. A qualifying binary allocation is not claimed. Finally, we identify an exact control-flow error in the incoming scratch-biased short-transfer experiment. We have not obtained a verified 214-state machine.

Contents

1 Results, scope, and conventions **3**

2 The accepted baseline and its mathematical interface **3**

3 Conditional origin markers **4**

4 The concrete 215-state construction **5**
4.1 Why the apparently incompatible caller is safe 5
4.2 The six changed transition rows 6

5 Consistency equivalence and the extent of the proof **6**

6 Fresh implementation and independent evidence **7**
6.1 A reproducible test of material difference 8

7 A new complete compiler for a two-head transfer command **8**
7.1 The precise command equations 8
7.2 A constructor lemma and two exact macros 9
7.3 A strong compiler interface and all axiom positions 10
7.4 Generalization and detachment 11
7.5 Enumeration and the new two-head credit theorem 11
7.6 A direct 61-leaf implementation of the same transfer 12
7.7 Exact source transformation and verification boundary 13

8 A correction to the archived scratch-biased experiment **13**
8.1 The extra right child changes the jump 14
8.2 An all-natural erroneous-return theorem 14
8.3 Connection to the actual blank run 14

9 The attempted 214-state routes and their limits **15**

10 Further research questions **16**

11 Reproduction and artifact map **17**

1 Results, scope, and conventions

The principal deliverable is a complete new 215-state machine, rather than a proposed edit awaiting a runtime check. The accompanying dense table uses working labels $0, \ldots, 214$, initial state 0, and halt 215. The native presentation preserves the predecessor's labels, has initial state 0, dispatcher 84, and halt 254. These are exactly the same machine under the supplied bijection. The state count includes every initialization state. There is one two-way infinite tape, alphabet $\{0, 1\}$, an initially zero tape, an initial head position of zero, and only left and right moves.

Result	Established status
Exact CM_{215} table	215 working states; 430 total transition rows.
Comparison with K_{215}	Ordinary all-tape halting-equivalence proof from state 0.
Consistency calibration	Inherited accepted numerical-calculus theorem plus the new equivalence.
Literal runtime	All 6,176 arbitrary-natural primitive contracts passed afresh.
Blank initialization	All 95,714 transitions independently replayed in both presentations.
Two-head transfer language	New ordinary compiler and infinite protected-stack theorem; source artifacts supplied.
214 states	No qualifying machine established.
Proof-assistant endpoint	No newly compiled Lean or Rocq endpoint asserted.

The repository's current exact-215 Lean and Rocq sources are explicitly marked as written but not kernel-checked in the inspected revision [1]. We retain that distinction. Our local equivalence proof is short enough to formalize separately, but a Python mathematical checker is not a Lean or Rocq kernel.

The inherited source, mathematical calibration, and implementation are separate objects. A complete source implementation theorem establishes that a physical table executes a specified numerical program. A calibration theorem establishes that this program searches precisely for the relevant refutations. Section 8 illustrates why both must be checked.

2 The accepted baseline and its mathematical interface

The pinned baseline is the native table with SHA-256

```
9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2
```

The unmodified source has SHA-256

```
17af207195179b2508bf91154cd00739374f8862207ca21b9a3cb9098aa6f5ba
```

It contains 4,751 primitive occurrences, 202 distinct source subtrees, and 6,176 guarded primitive cases. Its ordinary proof combines the dominated numerical calculus, a strong command compiler with cofinal proof codes, a whole-tape binary implementation, and a finite blank startup [2, 3].

For orientation, the numerical constructors use Cantor pairing

$$P(a, b) = \frac{(a + b)(a + b + 1)}{2} + b, \qquad I(a, b) = 4P(a, b), \qquad Q_a(b) = 4P(a, b) + 2.$$

Truth is code 0, falsity is code 1, and the alias $I(0,0) = 0$ is retained. Other native constructors and the exact gated theory bundle are unchanged. The eighth logical row is

$$L(A,B,C,U) = I\big(I(I(C,U),A),\ I(I(A,C),I(B,C))\big), \qquad U \le A.$$

The inherited finite derivation recovers unrestricted instances of this row from the dominated instances, and the inherited native-calculus comparison calibrates refutation with syntactic inconsistency of ZFC [4]. None of these calibrations is replaced by a new semantic interpretation of arbitrary formula codes.

For the principal machine result we need only the following accepted interface, with the exact baseline bytes fixed:

$$\mathsf{ZFC} \vdash \big(\mathrm{Halt}(\mathsf{K}_{215}, \mathrm{blank}) \leftrightarrow \neg\, \mathrm{Con}(\mathsf{ZFC})\big). \tag{1}$$

We transfer this statement by an all-tape machine equivalence. Thus the principal result does not depend on the new two-head language or on any experimental scheduler described later.

3 Conditional origin markers

A usual predecessor-marker fold requires the entire history of one state to have one predecessor bit and the entire history of another state to have the opposite bit. This condition can be weakened. If the two states already agree on the current symbol, their origin need not be recovered on that branch.

Lemma 3.1 (A conditional marker replacement)**.** *Let a deterministic binary machine T have distinct noninitial, nonhalting states p, q with*

$$T(p,0) = (0,R,a_1), \qquad T(q,0) = (0,R,a_0), \qquad T(p,1) = T(q,1).$$

Suppose that along every run from its specified initial state, on every initial tape, the following conditional invariants hold:

1. *at state p reading zero, the cell immediately to the left is one;*
2. *at state q reading zero, the cell immediately to the left is zero.*

Let x_0, x_1 be existing nonhalting provider states, distinct from p, q, with $T(x_i,0) = (0,R,a_i)$, and assume $a_i \neq q$. Construct N by redirecting old incoming edges to q into p, setting

$$N(p,0) = (0,L,q), \quad N(q,0) = (0,R,x_0), \quad N(q,1) = (1,R,x_1),$$

and replacing every non-overridden row (w,D,r) by $(w,D,\phi(r))$, where $\phi(q) = p$ and $\phi(r) = r$ for $r \neq q$. This rule includes the common read-one row at p. The displayed provider rows are therefore retained, and the new internal helper states are not halts. Then T and N halt equivalently from their specified common initial state, on every initial tape.

Proof. Relate complete configurations by the same tape and head and by the state map ϕ. Every ordinary old transition is simulated by one new transition, including the common read-one transition of p and q. On the exceptional branch, let the current head be at h, the current cell be zero, and the predecessor marker be i. The new machine executes

$$(p,h) \longrightarrow (q,h-1) \longrightarrow (x_i,h) \longrightarrow (a_i,h+1).$$

The three writes reproduce the values already present: zero, the marker i, and zero. Every other cell is untouched. The conditional invariant chooses exactly the endpoint of the old transition. Therefore one old step has the same full-tape effect as either one or three new steps.

Induction on old time constructs a concatenated new computation from the same initial configuration. Each simulation block has strictly positive length at most three. A finite old halt is reached by the new computation. An infinite old computation supplies an infinite new computation; by determinism this is the actual new run, so it cannot halt earlier. This also gives the reverse implication. The argument is valid for arbitrary tapes and does not inspect any cell outside the two-cell local block. □

The lemma does not require a one-to-one map on the old control states. The old label q denotes a decoder role before the transformation and an internal helper after it. This is why the construction is state-neutral: one role is merged, and the released label supplies the helper. A state saving would additionally require an existing compatible helper.

Corollary 3.2 (Exact time transfer)**.** *For every finite old computation with t transitions, let z count its visits to the two old states on the read-zero branch. The corresponding new computation has exactly $t + 2z$ transitions. In particular its length lies between t and $3t$.*

4 The concrete 215-state construction

Use $p = 52$, $q = 176$, $a_0 = 168$, $a_1 = 154$, $x_0 = 185$, and $x_1 = 129$. The baseline rows are

	0	1
52	$0R154$	$1R150$
176	$0R168$	$1R150$

and the providers satisfy $185/0 = 0R168$ and $129/0 = 0R154$. Here wDq denotes writing w, moving in direction D, and entering q.

4.1 Why the apparently incompatible caller is safe

The complete incoming-edge census is finite and exact:

Old destination	Incoming row	Information established
52	$190/1 = 1R52$	Left predecessor is one.
52	$193/1 = 1R52$	Left predecessor is one.
176	$165/0 = 0R176$	Left predecessor is zero.
176	$193/0 = 0R176$	Left predecessor is zero.
176	$156/1 = 1R176$	Current cell is one, by the preceding row.

The last row cannot simply be ignored: its predecessor marker is one, which would violate an unconditional origin invariant for state 176. However, the *only* incoming edge to state 156 is

$$189/1 = 1L156.$$

It writes one at a cell, steps left, and, if followed by 156/1, returns right to that same written cell. Consequently this arrival at 176 always reads one. The two old roles agree there. Every arrival at 176 that actually reads zero must instead come from 165/0 or 193/0, and has predecessor zero. State 52 always has predecessor one. Neither state is initial. This proves both premises of Lemma 3.1 for every run from state zero on every tape.

4.2 The six changed transition rows

All rows other than the following six remain byte-for-byte identical in the native tables:

State	Read	Baseline	New machine
52	0	$0R154$	$0L176$
156	1	$1R176$	$1R52$
165	0	$0R176$	$0R52$
176	0	$0R168$	$0R185$
176	1	$1R150$	$1R129$
193	0	$0R176$	$0R52$

In the new machine, the zero-marker block is

$$52 \longrightarrow 176 \longrightarrow 185 \longrightarrow 168,$$

and the one-marker block is

$$52 \longrightarrow 176 \longrightarrow 129 \longrightarrow 154.$$

Both leave the whole tape unchanged and move one cell right, precisely as the respective old read-zero row did. Ordinary decoder visits to 185 and 129 retain their original behavior. Their displayed zero rows are not modified.

Theorem 4.1 (Exact all-tape equivalence)**.** *For every initial binary tape τ with head at zero,*

$$\mathrm{Halt}(\mathsf{CM}_{215}, 0, \tau) \quad \Longleftrightarrow \quad \mathrm{Halt}(\mathsf{K}_{215}, 0, \tau).$$

The simulation has the exact time relation in Corollary 3.2.

Proof. The incoming census establishes the conditional invariants, the provider rows establish the two local blocks, and every remaining transition commutes with $\phi(176) = 52$. Apply Lemma 3.1. □

This theorem concerns the designated initial state. It deliberately does not quantify over arbitrary starts inside a merged role with an invalid marker. In particular, starting the new helper label 176 as if it were the old decoder role would describe a different initial configuration.

The complete native and dense tables are supplied in the archive. Their SHA-256 hashes are respectively

```
e5907f08e0752eada0e25bbf33680341e80e5fe94b52b409aea1ed039edda252
1f4f084f7293e2b042d5746039f793ff72ed429083b504970c939a5515e666cf
```

Every working state is reachable in the finite directed transition graph. This graph reachability statement is not a claim that every state is visited on the blank run.

5 Consistency equivalence and the extent of the proof

Theorem 5.1 (The requested consistency statement)**.** *For the exact supplied 215-state machine,*

$$\mathsf{ZFC} \vdash \left(\neg\, \mathrm{Halt}(\mathsf{CM}_{215}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{ZFC})\right).$$

Proof. Blank-tape halting for a fixed finite transition table and its finite computation codes are arithmetically definable. Arbitrary tape functions and their finite updates are also directly definable in ZFC. The finite incoming-edge checks and the two local tape calculations are formalizable in ZFC. Ordinary induction proves the configuration simulation, and positive block lengths prove preservation of halting and nonhalting. Hence Theorem 4.1 is formalizable in ZFC. Combine its blank instance with the accepted calibration (1) and apply classical negation. □

The consistency of ZFC is not an assumption used to prove this *equivalence*. If ZFC is consistent, the machine does not halt; if a finite ZFC refutation exists, it does halt. We make no unconditional consistency or nonhalting assertion and no minimum-state claim. We also make no unqualified two-sided independence claim: consistency assumptions for different unprovability directions must be stated separately.

The new proof depends on the pinned baseline's ordinary calibration. It does not re-prove the entire inherited native-calculus development. Copies of its relevant expositions, exact input tables, source, hashes, notices, and the fresh local evidence are included to make that dependency reviewable.

6 Fresh implementation and independent evidence

The source program is unchanged. A fresh run of the inherited symbolic checker on the new table proves every primitive case with arbitrary natural primary registers, arbitrary natural gap, and an arbitrary nonempty finite list of natural spare-block values. Unary scan rules are justified by induction on unary length, and spare-block traversals by list induction. The representation includes the complete tape and zero exterior. These are unbounded mathematical contracts; the time limit on constructing the finite certificates does not bound represented register values.

Quantity	Baseline	CM_{215}
Working states	215	215
Transition rows	430	430
Distinct source subtrees	202	202
Primitive occurrences	4,751	4,751
Guarded primitive cases	6,176	6,176
Symbolic proof rules	1,698,790	1,701,542
Minimum constant duration	6	6
Blank prefix length	95,714	95,714

An independent reader checks the ordered case indices and guards, every saved row reference, the exact table/source hashes, and full row coverage. All 430 rows are accounted for. Compared with the baseline's case records, 5,632 affine durations are identical, 128 increase by two, and 416 increase by six. Every variable coefficient and every stated motion remains identical. Thus 544 arbitrary-natural primitive cases actually use the changed mechanism. The new table is not merely a modification to an unobserved transition.

The complete blank trace is independently replayed against both native and dense tables. Its terminal frame is

$$\begin{aligned}&\text{native state } 84, \quad \text{head } -8, \quad \text{header } 57, \quad \text{gap } 1,\\ &(N,T,W,A,D,H,X,Y,L) = (4,0,0,0,0,0,0,0,0),\end{aligned}$$

with forty unit-valued spare blocks, 95 one-cells, and zero elsewhere. The new marker-zero branch is not used in this particular finite startup; that explains the identical prefix length. The arbitrary-natural runtime cases and the all-tape proof cover its later uses.

The ordinary equivalence theorem could transfer the accepted correctness without a second full primitive proof. The fresh proof and independent record audit are nevertheless useful here because the artifact requested is an exact new table, and they check both its emitted bytes and its compatibility with the existing verification pipeline.

6.1 A reproducible test of material difference

The new graph has one additional left-moving transition and one fewer right-moving transition than the baseline. Consequently no state renaming can identify them. The rooted graphs, including read symbols, written symbols, directions, and the halt, were additionally compared with 77 unique 215-state tables drawn from the checked-out candidate directory and all twelve incoming archives. None agrees, either directly or after left/right reflection. The inventory and comparison receipt retain exact hashes. This is a bounded novelty statement about that explicit corpus, not a claim to have searched every historical machine.

There is a concrete structural change: old roles 52 and 176 now share one boundary state, while label 176 becomes a predecessor-reading helper. A useful future allocator may be able to share that helper with another role. The present construction itself retains 215 states.

7 A new complete compiler for a two-head transfer command

The theory-side contribution is a three-command language whose transfer operation consumes two old stack heads at once. A deliberate use of the native zero implication supplies the missing parity control. The proof below establishes a complete command compiler and the required infinite stack-credit bound. A physical allocation, its arbitrary-natural runtime certificate, and its exact blank initialization remain separate obligations.

The inherited basis is the numerical calculus and emitter of accepted K215, the separately reviewed dominated-ρ_8 calibration, and the incoming MP0-transfer215 source. No new proof of the full native-calculus/ZFC interpretation is claimed in this section.

7.1 The precise command equations

Write

$$P(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad I(a,b) = 4P(a,b), \qquad Q_i(a) = 4P(i,a) + 2.$$

Truth is 0 and falsity is 1. Every parameter is a natural number, with no formula-decoding premise. Put $[x;S] = P(x,S)$, and write a work state as (t,S,d,h): current formula, stack, additive parameter, and cache. In particular $I(0,0) = 0$ and $[0;0] = 0$ are actual numerical identities.

The retained command $\mathsf{MP}(x)$ sets $y = d + x$. If $I(t,y) = h$ it returns $(y, [0;S], y, 0)$; otherwise it returns $(0, S, y, 0)$. The retained command $\mathsf{AX}(c;x,y)$ sets

$$U = d, \qquad A = d + x, \qquad B = h + y, \qquad C = c$$

and returns

$$\big(G(A,B,C), [r_1, \ldots, r_7, L(A,B,C,U), Q_U(h), t; S], 0, 0\big).$$

The first seven rows, literal gated bundle G, and dominated row L are the inherited emitter. The gate is zero when $C = A$ or $C = B$ and is the positive native bundle otherwise. All nine newly generated stack entries and the gate differ from 1. The rows are native theorems; $Q_U(h)$ is a theorem when h is. Numerical domination $U \leq A$ holds automatically.

Replace the old stack commands by

$$\mathsf{S}_2 = \mathsf{oldDROP}; \mathsf{oldNIP}; \mathsf{oldNIP}; \mathsf{oldNIP}.$$

Writing $S = [x, y; V]$, its exact nonhalting equation is

$$\mathsf{S}_2(t, [x, y; V], d, h) = (h, V, d, y). \tag{2}$$

It halts exactly when $h = 1$, $t = 1$, or $x = 1$. The final popped value y is not tested at this step. These are the three inherited NIP tests, which successively see h, the pushed old current t, and x.

If current, cache, and all exposed stack heads are native theorems, every nonhalting command preserves that condition. Every halt exhibits theorem 1. Successful MP is exact native detachment, while failure yields truth. This proves soundness on theorem-safe states, including states containing nondecodable numerical codes.

7.2 A constructor lemma and two exact macros

The sixth retained row and the dominated row are

$$\begin{aligned} r_6(A, B) &= I(I(Q_A(I(Q_A(B), 1)), 1), B), \\ L(A, B, C, U) &= I(I(I(C, U), A), I(I(A, C), I(B, C))), \qquad U \leq A. \end{aligned}$$

For all these parameters,

$$I(r_6(A, B), 0) \neq L(A, B, C, U). \tag{3}$$

Indeed $r_6 > 0$, so the left side is positive. Equality with a nonzero right side, followed by Cantor injectivity at the outer implication, would force $I(I(A, C), I(B, C)) = 0$. The zero law for I gives $A = B = C = 0$. Domination then gives $U = 0$ and $L = 0$, a contradiction. Equality with a zero right side is already impossible. This argument explicitly handles the exceptional truth alias.

Let $D_0 = \mathsf{AX}(0; 0, 0)$ and define

$$\begin{aligned} H_2 &= D_0; \mathsf{S}_2^6, \\ \mathsf{PUSH} &= D_0; \mathsf{S}_2^4; \mathsf{MP}(0); \mathsf{MP}(0); \mathsf{S}_2; \mathsf{MP}(0). \end{aligned}$$

For arbitrary numerical inputs, H_2 either halts or returns

$$(t, [x, y; V], d, h) \longmapsto (t, V, 0, y).$$

Its exact halt condition is $t = 1$ or $x = 1$. After the first AX and five shifts the state is $(r_8, [x, y; V], 0, t)$. The sixth shift gives the claimed result; every earlier tested generated value is constructor-safe. Thus H_2 removes two old heads while preserving the current theorem.

The PUSH macro is total, even when incoming current or cache equals 1, and has the exact equation

$$\mathsf{PUSH}(t, S, d, h) = (0, [t; S], 0, 0). \tag{4}$$

After $D_0; \mathsf{S}_2^4$ the state is $(r_6, [Q_d(h), t; S], 0, r_8)$. The first MP fails by (3). The second succeeds by $I(0, 0) = 0$ and pushes zero. The shift removes that zero and $Q_d(h)$, leaving $(0, [t; S], 0, Q_d(h))$. The final MP fails by the disjoint quantifier and implication tags. It clears the cache. No old stack cell or saved current is tested during this macro.

7.3 A strong compiler interface and all axiom positions

For a theorem code X, require a finite block B_X satisfying

$$B_X : (t, S, 0, 0) \longmapsto \text{halt, or } (X, S, 0, 0) \quad \text{for all raw } t, S. \tag{5}$$

Early halt is an allowed completeness outcome. Soundness is asserted only from theorem-safe states, not from arbitrary raw theorem data.

Define

$$\mathsf{DISCARD} = \mathsf{MP}(1); \mathsf{AX}(1; 0, 0); H_2^5.$$

The first MP fails against cache zero, since $I(t, 1) > 0$. It gives $(0, S, 1, 0)$. The AX has $U = A = C = 1$, so its gate is zero. It supplies ten safe heads, ending with old current zero. Five pair cleanups remove exactly those heads and return $(0, S, 0, 0)$ without halting or reading S. This is the truth compiler.

Put $J(0)$ equal to the empty word and $J(u) = \mathsf{MP}(u)$ for $u > 0$, used after DISCARD. It sets the parameter to u and preserves the stack, with current and cache zero. The omitted zero case is essential: $\mathsf{MP}(0)$ deliberately pushes a zero and serves as our parity adjustment.

Fix (A, B, C, U) with $U \leq A$. For an even position $j \in \{2, 4, 6, 8\}$, use

$$\mathsf{DISCARD}; J(U); \mathsf{AX}(C; A - U, B); \mathsf{S}_2^{j/2+1}; H_2^{(8-j)/2}. \tag{6}$$

After the shifts, current is r_j and exactly $8 - j$ generated heads remain over S. Pair cleanup removes them while retaining r_j. The final cached value is the saved old current zero, even when no cleanup remains. Hence this is a strong compiler for r_j.

For odd $j \in \{1, 3, 5, 7, 9\}$ insert one zero below the future packet by MP(0) after DISCARD, and one above the packet afterward. The upper padding word is

$$\mathsf{PAD}_G = \begin{cases} \mathsf{MP}(0), & C = A \text{ or } C = B, \\ \mathsf{MP}(0); \mathsf{MP}(0), & \text{otherwise.} \end{cases}$$

The positive-gate case first clears the current by a failing MP; the next MP succeeds. Thus exactly one zero is pushed in either case. This is a metalevel choice on known numerical axiom parameters, not a new command. Use

$$\begin{aligned} \mathsf{DISCARD}; \mathsf{MP}(0); J(U); \mathsf{AX}(C; A - U, B); \mathsf{PAD}_G; \\ \mathsf{S}_2^{(j+3)/2}; H_2^{(9-j)/2}. \end{aligned} \tag{7}$$

The explicit packet immediately after padding is $[0, r_1, \ldots, r_9, 0, 0; S]$, of length twelve. The shifts put r_j on top and leave $9 - j$ heads, an even number. Cleanup removes these and leaves cache zero. This selects every odd row, including the quantified ninth position, with the exact interface (5).

All first seven native rows may choose $U = 0$. Dominated ρ_8 uses the specified U. A permitted bundle has the simpler strong compiler

$$\mathsf{DISCARD}; \mathsf{AX}(C; A, B); H_2^5.$$

The gate remains on top as its ten generated stack entries are removed. All pairing arguments use $\text{unpair}(P(x, v)) = (x, v)$; no separate nil/cons constructor or positivity of an explicit zero wrapper is assumed.

7.4 Generalization and detachment

Given B_X, the following is a strong compiler for $Q_i(X)$:

$$\begin{aligned}\mathsf{DISCARD};\mathsf{MP}(0);B_X;\mathsf{PUSH};\mathsf{MP}(0);J(i);\mathsf{S}_2;\\ \mathsf{AX}(i;0,0);\mathsf{MP}(0);\mathsf{S}_2^6.\end{aligned} \tag{8}$$

Before $J(i)$ the exact state is $(0,[0,X,0;S],0,0)$. The shift after $J(i)$ returns $(0,[0;S],i,X)$. It does not test X, which is its second popped head. The AX has $U=A=C=i$, zero gate, and ninth output $Q_i(X)$. The final MP supplies the upper parity zero; six shifts perform the $j=9$ odd selection and return $(Q_i(X),S,0,0)$. Every GEN operation in the underlying calculus is theorem-only; no open assumption is generalized.

Suppose B_M compiles $M=I(A,B)$ and B_A compiles A. A strong compiler for B is

$$\begin{aligned}\mathsf{DISCARD};\mathsf{MP}(0);B_M;\mathsf{PUSH};\mathsf{MP}(0);B_A;\\ \mathsf{PUSH};\mathsf{MP}(0);\mathsf{S}_2;\mathsf{S}_2;\mathsf{MP}(B);H_2.\end{aligned} \tag{9}$$

Immediately before the shifts, the state is

$$(0,[0,A,0,I(A,B),0;S],0,0).$$

The shifts give $(A,[0;S],0,I(A,B))$, or halt early at $A=1$. MP succeeds and returns $(B,[0,0;S],B,0)$. The final H_2 removes the two zeros, resets parameter and cache, and preserves B, or halts at $B=1$. The proof includes $A=B=0$ and the zero implication alias.

Finite derivation induction now supplies (5) for every theorem of the dominated native calculus. Combine this with the inherited finite dominated-row proof transformation and native-calculus calibration. If 1 has a derivation, $B_1;H_2$ halts from every $(t,S,0,0)$: its final cleanup tests the saved current 1 before reading any old stack head. An initial AX resets arbitrary numeric parameter and cache, extending this completeness word to arbitrary work data.

7.5 Enumeration and the new two-head credit theorem

The source uses $\mathsf{MP}=0$, $\mathsf{S}_2=1$, and $\mathsf{AX}(c)=c+2$. The field lists are $[0,x]$, $[1]$, and $[c+2,y,x]$, respectively. Right-fold Cantor pairing encodes a field list, and every nonempty continuation is one plus its field encoding. Positive continuation pointers are decremented before reading the next command. Thus every nonhalting candidate drains finitely. The extra closed NIP changes neither the scheduler nor the field readers.

Let r encode the continuation $B_1;H_2$. The fresh candidates

$$n_Y=P(2,P(Y,P(1,r)))\geq Y$$

decode as $\mathsf{AX}(0;1,Y)$ followed by that continuation. The prefix resets parameter and cache. These successful candidates are cofinal when a refutation exists, proving completeness for every finite cutoff.

Soundness above an arbitrary buried stack tail requires a new bound: assign conservative balances $0,-2,10$ to MP, S_2, and AX. Successful MP pushes can only improve the balance. Let f be the right Cantor projection, $R(0)=[]$, and $R(r)=W(r-1)$ for $r>0$, where $W(n)$ is the complete syntactic word of fresh candidate n. Then

$$W(P(a,y))=\begin{cases}\mathsf{MP};R(f(y)), & a=0,\\ \mathsf{S}_2;R(y), & a=1,\\ \mathsf{AX};R(f(f(y))), & a\geq 2.\end{cases}$$

There are at most $n+1$ commands in $W(n)$, so every prefix of $R(r)$ has balance at least $-2r$.

Let $b(n)$ be total candidate balance, $B(k) = \sum_{n<k} b(n)$, and $g(0) = 0$, $g(r) = b(r-1)$ for $r > 0$. The Cantor prefix-pullback lemma states that nonnegative initial sums of q imply nonnegative initial sums of $q \circ f$ and $q \circ f \circ f$. Indeed the sequence $f(n)$ concatenates $[0]$, $[0,1]$, $[0,1,2]$, and so on. Apply it to $q = g + 2$; its first k terms sum to $2 + B(k-1) + 2(k-1)$.

Put $T_s = s(s+1)/2$. Joint induction proves $B(k) + 2k \geq 0$, $B(T_s) \geq -2$, and nonnegative relative balance at every command prefix within each diagonal $s \geq 2$. Such a diagonal has $s-1$ AX-first words, then an S_2-first word, then an MP-first word. The first k AX words have total balance at least $8k$ by double pullback. Inside AX word y, the lower bound is

$$8y + 10 - 2f(f(y)) \geq 6y + 10 > 0.$$

Through the penultimate word it is at least

$$8(s-1) - 2 - 2(s-1) = 6(s-1) - 2,$$

and through the final word it is at least

$$6(s-1) - 2 - 2f(s) \geq 4s - 8 \geq 0.$$

Every pullback index is below the current diagonal start. The first three words are MP, S_2, MP, with cumulative balances $0, 0, -2, -2$. This establishes the base and completes the induction. More explicitly, the prefix bounds give $B(k) \geq B(T_s) \geq -2$ at each candidate boundary in the current diagonal, hence $B(k) + 2k \geq 0$ there; the diagonal end establishes $B(T_{s+1}) \geq -2$. The pullback sums use only already established earlier values. Their formula for the first k terms applies for $k \geq 1$, while the empty partial sum is zero.

The infinite schedule therefore has minimum conservative prefix balance exactly -2, also from cutoff $N = 1$. Ten initial theorem heads supply permanent margin eight above an arbitrary encoded suffix. Every S_2 consumes two explicit theorem heads; its three guards test theorem values. This is an infinite-prefix theorem, not an inference from a finite simulation.

7.6 A direct 61-leaf implementation of the same transfer

The 104-leaf composite body has a shorter source alternative,

$$\mathsf{S}_2^{\mathrm{dir}} = \mathsf{CLEAR}_T; \mathsf{COPY}_{H \to T}; \mathsf{oldNIP}; \mathsf{oldNIP},$$

where the copy preserves H. Its nine-leaf prefix is inherited from the incoming direct one-head transfer implementation; the two-head use and complete stride-two compiler are new here. The literal prefix is

```
CLEAR_T = (CD1, CC1)
COPY_H = ((CD5,(CI1,(CI6,CC3))), (CD6,(CI5,CC2))).
```

Registers 1, 5, and 6 are current T, cache H, and scratch X. At an inherited command boundary $X = 0$. Clearing T takes $2t + 1$ source steps. The first copy loop performs h decrements of H, incrementing T and X, and ends at $(T, H, X) = (h, 0, h)$ in $4h + 1$ steps. The second transfers X back into H in $3h + 1$ steps. Thus ordinary induction on the counters gives the total prefix contract

$$(t, S, d, h; X = 0) \longmapsto (h, S, d, h; X = 0)$$

in $2t + 7h + 3$ source steps, preserving every other register. The formulas include $t = 0$ and $h = 0$.

OldNIP tests its incoming cache, and on a nonhalting return pops one stack head into that cache while preserving current and parameter. Two such calls therefore give

$$\mathsf{S}_2^{\mathrm{dir}}(t,[x,y;V],d,h)=\begin{cases}\text{halt}, & h=1 \text{ or } x=1,\\ (h,V,d,y), & \text{otherwise.}\end{cases}$$

Only the composite's old-current test is omitted. No two-way behavioral equivalence is asserted. All displayed normal-return equations and stack balances are identical, so the strong compiler still applies. The essential terminal test remains: after D_0 and five direct shifts the state is $(r_8,[x,y;V],0,t)$. The sixth tests saved current t through its cache. Hence H_2 still halts exactly when $t=1$ or $x=1$, and $B_1;H_2$ still detects every returned refutation. PUSH and DISCARD remain total; the two shifts in the MP compiler still test A through cache, and the final H_2 still detects $B=1$.

The five direct source associations have 202 distinct subtrees, 4,758 primitive occurrences, and 6,187 guarded cases, with the same depth 44 and maximum zero run six. Their command body contains $2+7+26+26=61$ leaves instead of 104, saving 43 primitive occurrences. This is a source-size result. It does not establish a physical state reduction. The same infinite credit proof applies because the command fields and the two-head stack consumption are unchanged.

7.7 Exact source transformation and verification boundary

The pinned parent source has SHA-256

`8ddb942f0f1d8e2e3b12ab7217c7232eb553cd288fca38dbf9b0eded73893dd5.`

At address `110101`, replace its $((D,N),N)$ by a binary association of D,N,N,N. Each old body has 26 primitives and zero escaping break/continue demand. NIP contains an intentional halt, which is retained. The five supplied associations each have 202 distinct subtrees, 4,801 primitive occurrences, 6,245 guarded cases, maximum depth 44, and maximum zero run six. The source builder checks both complete CFG resolvers and exact transport of all 4,697 exterior instructions and 6,103 exterior edges.

The stack checker passes 79 interface cases, with totality enforced for PUSH and DISCARD and mandatory halting enforced for the designated H_2 tests. It independently evaluates 11,232 numerical instances of the constructor lemma. The credit recurrence is corroborated for 100,000 candidates and 444 complete diagonals. A separate literal source checker evaluates 81 DNNN register cases and confirms exactly the three claimed guards, with 57 guarded halts. Its direct-command option checks 81 further cases and confirms the two direct guards, with 45 guarded halts. Both source forms pass the same 79 interface checks and the same constructor and credit corroborations. These finite checks support the written all-natural proof; they do not replace it. Inductive premise-block placeholders stand for (5), not asserted machine transitions.

An exact binary result additionally requires a literal transition table, an initialization-inclusive state count, every arbitrary-natural primitive runtime case for this exact source, and a complete blank trace to an admissible frame. The construction and proof in this section do not alone establish those physical obligations.

8 A correction to the archived scratch-biased experiment

The incoming `T215_Transfer_2026-10-20` packet contains an experimental short-transfer table with 215 states and a complete quantified primitive-runtime certificate. Its logical calibration

was explicitly left open. The accompanying description of its scheduler is incorrect: a source reassociation changes the destination of the final backward jump. This does not affect the accepted composite-transfer machine in that packet.

The affected native table has SHA-256

`f08bed9e62b9f43237b65c465d2f1bfc17474637b4b02785ca7f1813827bb0ce`

and the source has SHA-256

`2407f815551b4aba335c9bf041e15e8ce3cfb9ad830d89539f3bcf123462371a`

These identities select the research object precisely; no primary candidate file is altered by this audit.

8.1 The extra right child changes the jump

In this structured source language, CC(k) cuts its binary address immediately before the kth one encountered from the right and resumes at the first leaf below that prefix. Before insertion, the return leaf was `CC3` at address `10011`, which cuts to the empty prefix. Replacing that leaf by (CI6, CC3) puts the final leaf at `100111`. This address has four ones. Cutting at the third one from the right now retains prefix `100`, whose first leaf is source ordinal 23, the command selector. It does not return to scheduler ordinal 0.

The literal source rows are

$$4696:\ \mathrm{CI6}, \qquad 4697:\ \mathrm{CC3} \longrightarrow 23, \qquad 23:\ \mathrm{CD3}.$$

Changing only the source operand to `CC4` would restore target 0, but that is a different source. Its distinct-subtree count is 203, compared with 201 for the erroneous source, and no same-size physical repair follows merely from retagging it.

8.2 An all-natural erroneous-return theorem

Write the registers in the order $(N, T, W, A, D, H, X, Y, L)$. At ordinal 4696 suppose $A = H = X = 0$; the remaining six registers are arbitrary natural numbers. Then the literal source reaches its halting primitive, ordinal 4718, after exactly $2T + 15$ nonhalting primitive transitions.

To prove this, the first instruction sets $X = 1$. The incorrect jump enters the selector, where $A = 0$ routes to the short-transfer body. Its clear loop sets $T = 0$ in $2T + 1$ steps. The copy of H into T executes zero loop iterations because $H = 0$, but its restoring loop transfers the residual $X = 1$ into H. Thus $H = 1$ and $X = 0$. The inherited NIP guard then tests this manufactured 1 and enters its halting primitive. The stack W is not read. The remaining literal path has fixed length, giving $2T + 15$. The independent script supplies the complete 15-step witness for $T = 0$ and checks the general formula on a separate finite grid; the argument above is the all-natural justification.

8.3 Connection to the actual blank run

The independent audit replays all 69,744 binary transitions in the archived blank prefix. It reaches native dispatcher 84, head -2, at source ordinal 64 with registers

$$(1, 4, 0, 0, 0, 0, 3, 10, 0).$$

An independently resolved source CFG reproduces this frame after 172 primitive transitions from AX entry 25 and seed

$$(1, 0, 3, 0, 0, 0, 0, 0, 0).$$

The entire AX interval $[25, 4590)$ has exactly the same ordered primitive word and ordinal control-flow graph as the pinned parent. Its inherited all-natural termination contract therefore applies. At this AX seed, $d = x = c = 0$, so $A = d + x = c$ and the gate is $T = 0$; it exits at ordinal 4696 with $A = H = X = 0$. The preceding theorem then reaches CH in 15 further nonhalting source transitions. Executing that CH halts the binary machine, by its separately supplied complete primitive-runtime theorem.

This is an eventual-halting proof using the inherited AX contract and quantified binary simulation. We have not executed the entire enormous unary AX computation or claimed an exact total binary halting time. The concrete 69,744-step blank prefix and the 15-step source suffix are separately replayed. The experiment consequently cannot serve as a nonhalting consistency witness under Con(ZFC).

The exact safe-stack observation remains useful for a future corrected scheduler: $W = 3 = P(2, 0)$ and $2 = Q_0(0)$ is a theorem by generalization of truth. Thus this particular seed has a theorem-valued entire stack; it does not require a protected-unsafe-tail estimate. That observation does not repair the wrong control-flow edge.

9 The attempted 214-state routes and their limits

We investigated both source packing and direct binary changes before selecting the verified result. Source DAG counts were used as a search measure, never as a substitute for a physical state count.

A closed-reassociation beam with the existing partial selector frozen examined 801 sources and retained a best count of 202 subtrees. A wider beam that allowed changes inside its unused children examined 94,227 sources in a bounded run and found 202-subtree presentations with depth 41 instead of 47. Four representative attempts to allocate these shallower sources into the existing protected row pool were infeasible under that specific allocation policy. This does not rule out a different allocator.

Two further beams allowed the more general association $(A, (B, C)) \leftrightarrow ((A, B), C)$ whenever A has no escaping forward jump and C has no escaping backward jump; the middle block may be open. Every saved result was checked by both full source CFG resolvers. The bounded runs examined 69,868 and 106,432 source trees. Their best saved value of "DAG nodes plus twice maximum zero run" was 214, corresponding to 202 nodes and zero-run length six. This numerical score is a source-search score, *not* a 214-state result.

Deleting native scanner state 50 and redirecting 34/0 to 75 produces a 214-state table and a calibrated finite blank prefix, but the complete arbitrary-natural runtime attempt does not pass. Of the 6,176 cases, 6,104 pass and 72 cases at 37 source sites fail to be certified. Those sites are exactly the addresses ending in six zeros.

The archive gives an actual counterexample, beyond this failure to derive the contracts. At source ordinal 26, the primitive is CD(1) at address `1101001001000000`. Take all nine logical registers zero, gap zero, spare list [0], native dispatcher 84, and head zero. The full initial tape support is

$$\{-1, 0, 1, 3, 6, 9, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84\}.$$

The baseline reaches its first clean dispatcher frame after 588 steps, with the required address `110100100100001`. The shortened table reaches its first clean dispatcher frame after 662 steps, but

its address is `1101001001001`: one cell at position 14 has moved to position 12, while all other one-cells agree. Both complete traces are independently replayed by the supplied counterexample checker. This disproves the proposed arbitrary-frame implementation contract. It does not establish a separate claim about that table's blank-run consistency equivalence. Section 10 makes the remaining source/interface question explicit.

The two-head transfer source has 202 subtrees. In the first direct protected allocation, eight changed roles competed for seven usable existing role slots, and the model was infeasible. The other source associations and possible global reallocation remain research. This is not a physical lower bound on the new formalism.

The state-neutral conditional-marker construction was chosen because it has a complete local theorem and passes the exact implementation checks. We do not relabel a failed 214-state search as a successful reduction.

10 Further research questions

1. **Can the new helper be shared?** The old decoder role 176 now occupies the boundary state 52. The released label implements $0R185/1R129$. An allocation that supplies that selector with another existing state would turn this particular construction into a 214-state one. Its caller-context and ordinary decoder obligations must be checked together.

2. **Can two conditional folds use one helper?** Separate safe folds can be state-neutral, while a common helper might make their composition save a state. The concrete marker relations give a small finite search target; the helper must avoid recursive dependence on either merged role.

3. **Can six-zero return suffixes be eliminated at no net cost?** The failed scanner shortening isolates 37 source addresses. A semantics- preserving reparse, a different control encoding, or a contextual bypass that repairs those exact returns could make the deletion of state 50 useful. A smaller maximum source depth alone is insufficient.

4. **How far does two-head transfer generalize?** The new proof uses successful $\mathsf{MP}(0)$ as explicit parity padding and a non-detachment identity to recover a total PUSH. For transfers that remove k heads, what finite padding and packet-selection conditions yield an equally strong compiler? What is the actual binary cost of those languages?

5. **Can the two-head source be allocated within 215 states?** Its 202-node source count is close to the accepted source count, but contextual engine overlaps matter. The five exact source variants give concrete starting points for a role allocator or a different return encoding.

6. **Can marker proofs be synthesized with small checked certificates?** The successful invariant uses a two-transition fact about an anomalous caller. A finite analysis of neighboring tape cells could synthesize such guarded origin assertions, then emit a small independent certificate of every incoming case.

7. **Can future source edits automatically certify return binding?** The short-transfer failure changes the target of a continuation when one branch bit is inserted. A source transformation should emit both old and new complete guarded CFGs, or a stated semantic relation, before any logical compiler theorem is transferred.

8. **Can the exact new table receive a compiled formal endpoint?** The all-tape conditional-marker lemma offers a short route from an eventual checked K_{215} endpoint. It should be formalized at the literal-table level, with the two-step caller guard and the positive simulation-block lengths made explicit.

11 Reproduction and artifact map

The archive is intended to be unpacked as one directory. The main machine, source, literal six-row recipe, canonicalization map, baseline inputs, ordinary proof, symbolic checker, independently written audits, and compressed evidence are included. The main standard-library validation entry point is:

```
python3 -B verify.py
```

It verifies immutable input hashes, reconstructs both tables, checks the conditional-marker proof data, audits the saved arbitrary-natural cases, replays both complete blank traces, runs the two-head interface checks, and reproduces the return-target counterexample. It does not invoke an optimizer or download dependencies.

To rederive the full symbolic primitive proof in a new directory:

```
python3 -B verify.py --runtime /absolute/path/to/fresh-runtime
```

This optional step can take several minutes. It reruns the mathematical checker; the ordinary source/calculus calibration remains the explicitly stated proof dependency. The saved proof and complete runtime are bound to the exact table hashes printed above.

The document is built with ordinary pdfLaTeX:

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The root manifest lists every shipped file with its SHA-256. Runtime records are compressed losslessly. Failed searches and hypothetical corrected sources are kept under `research/`, apart from the accepted `machine/` directory.

Attribution and proof ownership

The accepted K215 table, its numerical calculus, dominated-row derivation, source compiler, original runtime checker, and preceding marker folds are inherited from the ProveIt contributors and the upstream work cited in the retained notices. This packet contributes the conditional 52/176 marker construction and its local proof, its fresh verification and independent comparison, the two-head transfer compiler and credit argument, and the explicit short-transfer return correction. The new source-side theorem is separate from the main machine calibration. Original licenses and notices are preserved. No authorship of the inherited 215-state calibration is claimed.

References

[1] ProveIt contributors, *K215: 215 working states for ZFC consistency*, inspected at commit `75716498437bfc7291d5900cab2983a11fd54bac`, 21 October 2026. Pinned candidate directory on GitHub.

[2] ProveIt contributors, *Ordinary proof of the combined 215 machine*, `candidate-k215/proof.md`, same pinned commit. A copy is included in `reference/`.

[3] ProveIt contributors, *Reproduction and evidence scope*, `candidate-k215/reproduction.md`, same pinned commit.

[4] ProveIt contributors, *Dominated-rho8 construction*, `candidate-k218/alternatives/dominated/proof.md`, same pinned commit.

[5] ProveIt incoming packet, *Complete proof compiler for the three-command NIP-transfer enumerator*, retained under `T215_Transfer_2026-10-20/reference/transfer-parent/proofs/`. This is a source of inherited interfaces, not a proof of the literal scratch-biased source corrected here.

[6] ProveIt incoming packet, *MP0 Transfer 215*, `docs/incoming/MP0_Transfer_215.zip`, same pinned commit. The exact source and native table hashes used for the two-head derivative are retained in its source-construction record.

[7] ProveIt incoming packet, *Short-transfer experiments: rejected implementations and one open 215-state machine*, `docs/incoming/T215_Transfer_2026-10-20.zip`, `research/short-transfer/`. The scratch-biased table and source are identified by full hashes in Section 8.